

22 Searching and Sorting - Zero to Hero questions list

July 18, 2026

1 Searching and Sorting in C — Practice Questions (50 Questions with Hints Only)

The questions are arranged from **Beginner** → **Intermediate** → **Advanced**.

2 Beginner Level (1–15)

2.0.1 1. Search for an element in an array using Linear Search.

Hint: Compare each element one by one until a match is found.

2.0.2 2. Count the number of occurrences of a given element.

Hint: Increment a counter whenever a match is found.

2.0.3 3. Find the first occurrence of an element.

Hint: Stop searching after the first match.

2.0.4 4. Find the last occurrence of an element.

Hint: Continue searching until the end of the array.

2.0.5 5. Check whether a given element exists in an array.

Hint: Use a flag variable.

2.0.6 6. Search for a character in a string.

Hint: Traverse the string until '\0'.

2.0.7 7. Search for a word in a string.

Hint: Compare substrings or use string functions.

2.0.8 8. Find the index of the largest element in an array.

Hint: Keep track of both the maximum value and its index.

2.0.9 9. Find the index of the smallest element in an array.

Hint: Update the minimum index while traversing.

2.0.10 10. Find both the maximum and minimum elements in one traversal.

Hint: Update two variables simultaneously.

2.0.11 11. Sort an array in ascending order using Bubble Sort.

Hint: Repeatedly swap adjacent elements.

2.0.12 12. Sort an array in descending order using Bubble Sort.

Hint: Reverse the comparison condition.

2.0.13 13. Sort a string alphabetically.

Hint: Compare and swap characters.

2.0.14 14. Sort an array containing only 0s and 1s.

Hint: Count occurrences or use two pointers.

2.0.15 15. Sort three numbers without using arrays.

Hint: Use conditional swaps.

3 Intermediate Level (16–35)

3.0.1 16. Implement Selection Sort.

Hint: Select the smallest element in each pass.

3.0.2 17. Implement Insertion Sort.

Hint: Insert each element into its correct position.

3.0.3 18. Compare Bubble Sort and Selection Sort.

Hint: Count comparisons and swaps.

3.0.4 19. Compare Selection Sort and Insertion Sort.

Hint: Test using the same input.

3.0.5 20. Implement Binary Search.

Hint: The array must be sorted first.

3.0.6 21. Perform Binary Search on user-entered sorted data.

Hint: Divide the search interval repeatedly.

3.0.7 22. Count the number of comparisons made during Binary Search.

Hint: Increment a counter in each iteration.

3.0.8 23. Find the insertion position for an element in a sorted array.

Hint: Use Binary Search logic.

3.0.9 24. Search for an element in a descending sorted array.

Hint: Reverse the comparison conditions.

3.0.10 25. Search for multiple occurrences of an element.

Hint: Find one occurrence, then scan neighbors.

3.0.11 26. Merge two sorted arrays into a third sorted array.

Hint: Use two pointers.

3.0.12 27. Find the median of a sorted array.

Hint: Handle both even and odd sizes.

3.0.13 28. Find the second largest element.

Hint: Maintain both largest and second largest values.

3.0.14 29. Find the second smallest element.

Hint: Maintain both smallest and second smallest values.

3.0.15 30. Find the kth largest element.

Hint: Sort the array or use repeated selection.

3.0.16 31. Find the kth smallest element.

Hint: Sort or partially sort the array.

3.0.17 32. Remove duplicate elements from a sorted array.

Hint: Use two indices.

3.0.18 33. Find common elements between two sorted arrays.

Hint: Traverse both arrays simultaneously.

3.0.19 34. Find the union of two sorted arrays.

Hint: Merge while avoiding duplicates.

3.0.20 35. Find the intersection of two sorted arrays.

Hint: Compare current elements of both arrays.

4 Advanced Level (36–50)

4.0.1 36. Implement Quick Sort.

Hint: Choose a pivot and partition the array.

4.0.2 37. Implement Merge Sort.

Hint: Divide the array into halves recursively.

4.0.3 38. Compare Merge Sort and Quick Sort.

Hint: Analyze time complexity and memory usage.

4.0.4 39. Implement Heap Sort.

Hint: Build a max heap and repeatedly extract the root.

4.0.5 40. Implement Shell Sort.

Hint: Reduce the gap after each pass.

4.0.6 41. Implement Counting Sort.

Hint: Suitable for a limited range of integers.

4.0.7 42. Implement Radix Sort.

Hint: Sort digits from least significant to most significant.

4.0.8 43. Implement Bucket Sort.

Hint: Divide elements into buckets and sort each bucket.

4.0.9 44. Search for an element in a rotated sorted array.

Hint: Determine which half is sorted in each step.

4.0.10 45. Find the first and last occurrence of an element using Binary Search.

Hint: Perform modified Binary Search twice.

4.0.11 46. Build a Student Record Search System.

Hint: Implement:

- Linear Search by Roll Number
 - Binary Search after Sorting
 - Search by Name
-

4.0.12 47. Build an Employee Management Search System.

Hint: Sort employees by ID or salary and perform searches.

4.0.13 48. Build a Phone Directory.

Hint: Sort contacts alphabetically and search by name or phone number.

4.0.14 49. Build a Library Catalog Search System.

Hint: Search books by:

- Title
 - Author
 - ISBN
 - Publication Year
-

4.0.15 50. Build a Comprehensive Searching and Sorting Toolkit.

Hint: Create a menu-driven program implementing:

- Linear Search
- Binary Search
- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort
- Counting Sort
- Radix Sort
- Display Performance Statistics (comparisons, swaps, execution time)
- Compare Time Complexities for Different Input Sizes